

Automated Vehicle Access Control System (FPGA-Based FSM Controller)

A PROJECT DOCUMENTATION

Presented to the Faculty of the
Department of Computer Applications
College of Computer Studies
MSU - Iligan Institute of Technology
Iligan City

In Partial Fulfillment
of the requirements for the Course
BCA181 - HDL PROGRAMMING

Shaqkobe Dos Tejada

Kent Paolo L. Quintana

Joshua I. Rocamora

Cassandra Khan P. Mendoza

May 2026

Abstract / Executive Summary

This project presents the design and simulation of an **Automated Vehicle Access Control System** implemented using a Verilog-based Finite State Machine (FSM). The primary objective is to automate parking barrier operations while integrating safety protocols to prevent mechanical and vehicle damage. By utilizing sequential logic and sensor-driven transitions, the system manages four distinct states: **IDLE**, **OPENING**, **OPEN**, and **CLOSING**. The design demonstrates high reliability in executing immediate safety reversals during obstruction events, ensuring the structural integrity of both the barrier and the vehicle. This implementation serves as a robust framework for real-world automated infrastructure applications.

TABLE OF CONTENTS

Title Page	1
Abstract / Executive Summary	2
Table of Contents (Summary)	4
List of Tables	4
Chapter 1: Introduction	4
1.1 Background of the Study.....	4
1.2 Objectives.....	4
1.3 Scope and Limitations.....	4
Chapter 2: Review of Related Literature	5
2.1 Finite State Machines (FSM) in Automation.....	5
2.2 Sequential Logic and Memory Foundations.....	5
2.3 Safety Protocols in Access Control.....	5
Chapter 3: Methodology	6
3.1 Hardware Logic Foundations.....	6
3.2 Conceptual Analogy: The Role of Memory in Automation.....	6
3.3 Synthesis into FPGA Architecture.....	8
Chapter 4: Results & Discussion	9
4.1 Simulation Results and Waveform Analysis.....	9
4.1.1 Critical Safety Cycle Verification.....	9
4.2 Test Cases and Boundary Conditions.....	10
4.2.1 Analysis of Conflicting Inputs (TC-05).....	11
Chapter 5: Conclusion & Recommendations	12
5.1 Conclusion.....	12
5.2 Recommendations.....	12
Appendices	14
Source Code:.....	14
A.1 Barrier Control FSM (Finite State Machine).....	14
A.2 UART Module (Proposed Hardware Interface Extension).....	14
A.3 PWM Module (Proposed Mechanical Driver Extension).....	14
Appendix B:.....	15
Datasheets.....	15
B.1 Synchronous Gating and Propagation DelaysFlip-Flop Timing:.....	15
B.2 Component Specifications (Proposed Hardware Interface).....	15

LIST OF TABLES

Table 1: FSM State Diagram.....	15
--	----

LIST OF FIGURES

Figure 1: Operational Logic Flowchart of the Parking Barrier FSM	7
Figure 2: Vivado RTL Schematic of the Top-Level Entity (gate_state_reg)	6
Figure 3: Cross-Coupled NOR SR Latch Conceptual Memory Circuit	6
Figure 4: Behavioral Simulation Waveform - Normal Operational State Cycle	9
Figure 5: Behavioral Simulation Waveform - High-Priority Obstruction Override	10

CHAPTER I

INTRODUCTION

1.1 Background of the Study

In modern urban environments like Iligan City, manual vehicle monitoring is increasingly inefficient. Manual gates are prone to human error and slow processing times, leading to traffic congestion at entry points. This project focuses on utilizing **Hardware Description Language (Verilog)** to create a high-speed, reliable automation system. By shifting from manual operation to a **Finite State Machine (FSM)**, the system ensures consistent performance and integrated safety protocols.

1.2 Objectives

The primary goal is to design a functional parking barrier controller that operates autonomously. Specific objectives include:

- Implementing a four-state **FSM** that prioritizes immediate state reversal from **CLOSING** to **OPENING** upon obstruction detection.
- Integrating an **obstruction sensor** as a high-priority safety override to prevent vehicle impact.
- Simulating the digital logic in **Xilinx Vivado** to verify timing and state transitions.

1.3 Scope and Limitations

- **Scope:** This study covers the digital logic design, RTL analysis, and behavioral simulation of the control unit.
- **Limitations:** The project focuses on the logic controller; it does not include the physical construction of the barrier hardware or the specific motor torque calculations.

CHAPTER II

REVIEW OF RELATED LITERATURE

2.1 Finite State Machines (FSM) in Automation

Finite State Machines are the industry standard for sequential control systems. Literature shows that Mealy and Moore machines provide a predictable framework for automation, ensuring that a system transitions only when specific logical conditions—such as a payment confirmation—are met.

2.2 Sequential Logic and Memory Foundations

At the gate level, automation relies on the ability of a circuit to store previous state information. While fundamental asynchronous storage elements like the cross-coupled NOR SR Latch provide the foundational theory of hardware memory, modern FPGA architectures utilize clock-edge-triggered D-Flip Flops. Research indicates that synchronous memory elements prevent hazards and race conditions, allowing a system to securely maintain its state (such as an open gate command) relative to a global clock domain without requiring continuous sensor activation.

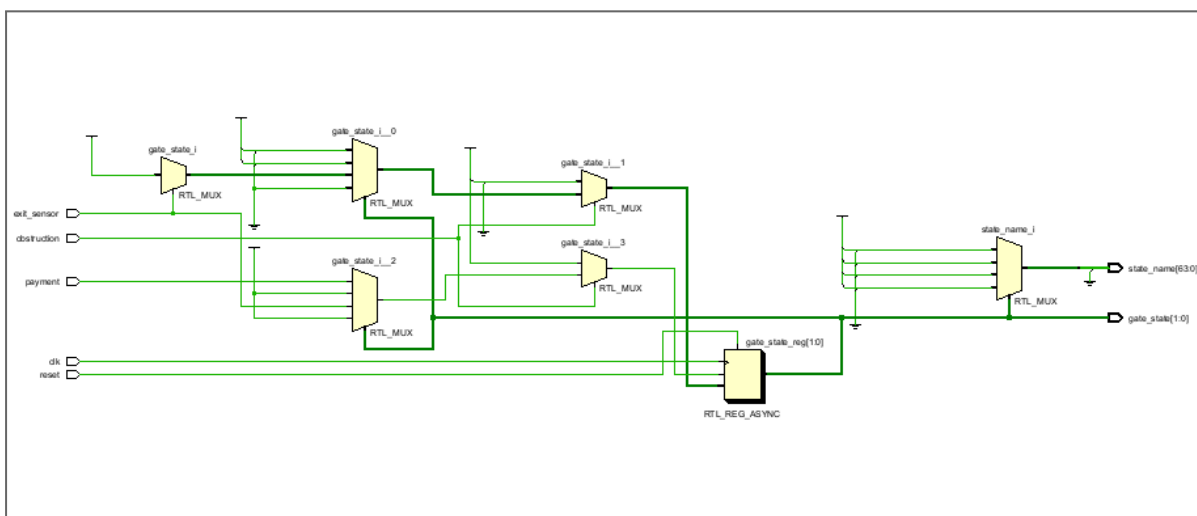
2.3 Safety Protocols in Access Control

Previous studies in automated infrastructure emphasize the necessity of "fail-safe" mechanisms. The use of **asynchronous reset signals** or high-priority obstruction inputs is a documented method for preventing mechanical failure. This project builds on these concepts by prioritizing the obstruction input to override the CLOSING state, ensuring the system preserves the structural integrity of both the barrier and the vehicle.

CHAPTER III METHODOLOGY

3.1 Hardware Logic Foundations

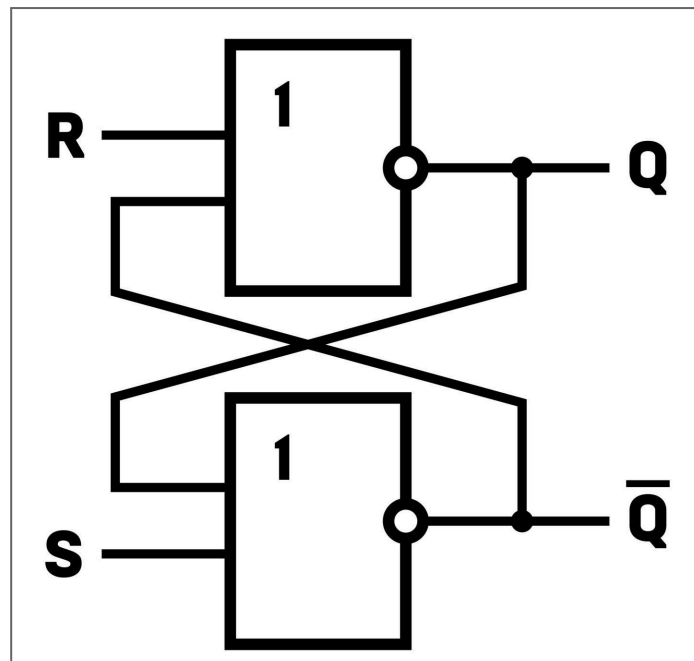
The core of the control unit relies on synchronous sequential memory structures to track system states. While conceptual designs often reference Set-Reset (SR) latch principles for basic state retention, this FPGA-targeted control unit implements edge-triggered registers to prevent timing glitches.



3.2 Conceptual Analogy: The Role of Memory in Automation

The system utilizes clock-synchronized storage to achieve state permanence and structural reliability:

- **State Persistence:** When a payment signal is asserted, the synchronous logic registers the input. On the next rising clock edge, the state register locks onto the OPENING state value, ensuring the system continues to process the operation even after the initial payment pulse drops.
- **Synchronous Feedback:** The internal state tracking eliminates the need for continuous sensor activation while a vehicle is clearing the gate.
- **Safety Priority Execution:** The obstruction loop acts as an absolute conditional override. If triggered during a closing sequence, the next state generation logic instantly bypasses normal execution variables, routing the binary code for OPENING directly into the D-Flip Flop input gates.



Why It's Critical

Without this "memory" logic, your barrier would be "forgetful" ; it would stop moving the exact millisecond a sensor stopped sending a signal. The SR Latch provides the automation needed to keep the gate moving until the next specific event occurs.

The "Memory" Logic Example

Imagine the **SR Latch** is the "brain" of your barrier motor:

- **The "Start Opening" Signal:** When a user pays, a brief pulse is sent to the **Set (S)** input. The latch "remembers" this high signal and keeps the motor running even after the payment process is finished.
- **The "Stop/Hold" Signal:** The feedback loop (the crossing lines in the image) keeps the **Q** output active. This allows the barrier to stay in the **OPEN** state without requiring the sensor to stay active the entire time a car is passing through.
- **The "Safety/Reset" Signal:** If the **obstruction** sensor or a limit switch is triggered, it sends a pulse to the **Reset (R)** input. The latch "remembers" that the movement must reverse, overriding the previous "Opening" instruction to prevent mechanical damage.

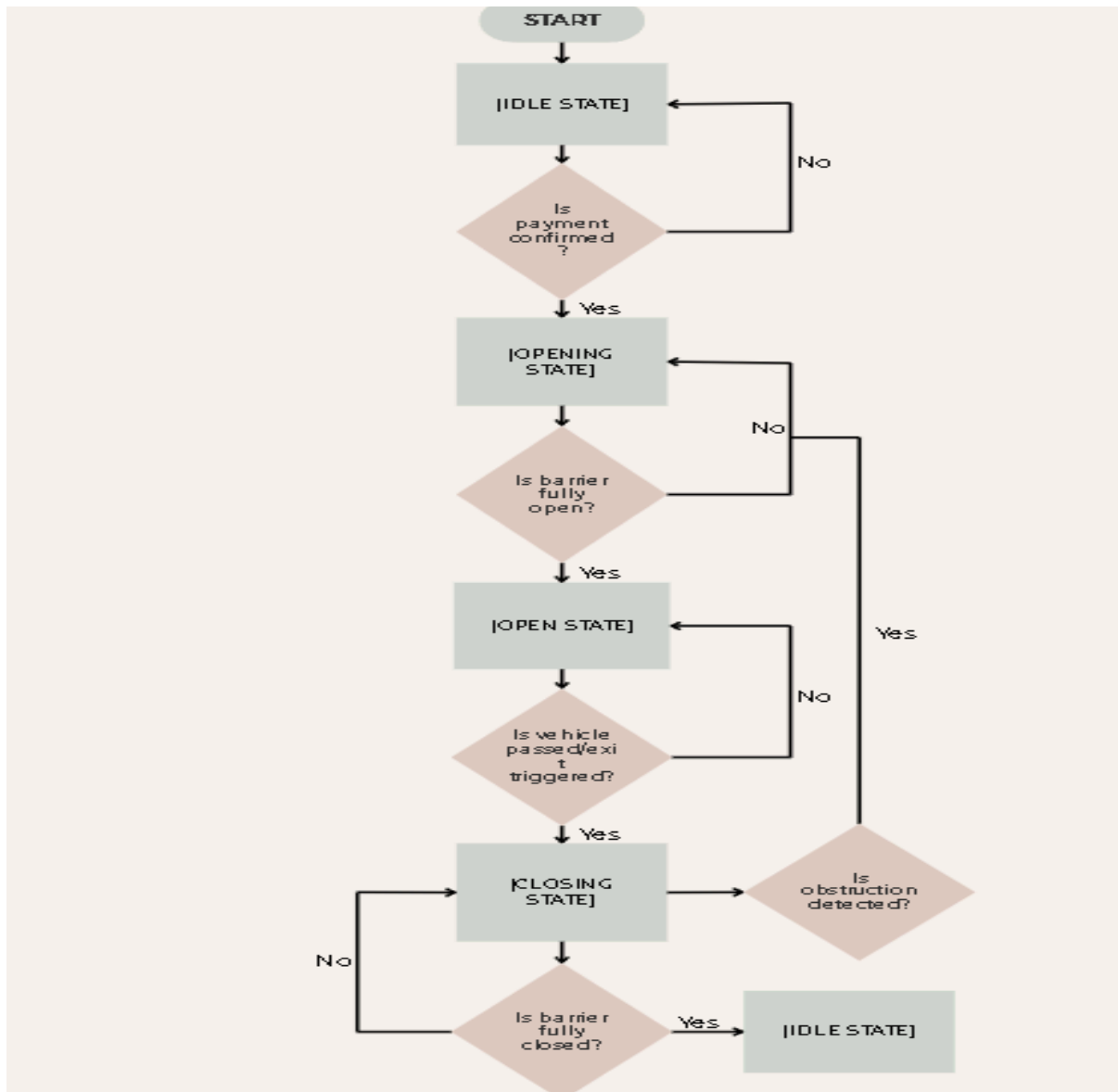


Figure 1: Operational Logic Flowchart of the Parking Barrier FSM

3.3 Synthesis into FPGA Architecture

During the synthesis process in Xilinx Vivado, these behavioral Verilog descriptions are mapped onto physical FPGA resources:

1. **D-Flip Flops:** Used for synchronized state storage tied to the system clock.
2. **Look-Up Tables (LUTs):** Used to implement the combinational logic gates (AND, OR, NOT) that define transition rules.
3. **Multiplexers (MUXes):** Used to route the "Next State" logic into the memory registers.

CHAPTER IV

RESULTS AND DISCUSSION

4.1 Simulation Results and Waveform Analysis

The behavioral logic of the automated parking barrier was validated using Xilinx Vivado. Simulation waveforms confirm that the system handles synchronous state transitions relative to the system clock heartbeat while safely evaluating external asynchronous sensor loops.

4.1.1 Critical Safety Cycle Verification

The most vital safety feature verified during testing is the Obstruction Override sequence.

- **Observation:** When the Finite State Machine (FSM) is in the CLOSING (2'b11) state, the system is actively driving the barrier downward.
- **Trigger Event:** Upon asserting the obstruction sensor signal to high (logic '1'), the system registers a safety hazard under the gate beam.
- **Result:** As shown in the simulation timing waveforms, the FSM instantly abandons the closing sequence and shifts directly to the OPENING (2'b01) state on the next immediate rising edge of the system clock.

Timing Calculation: With a clock period configured for a 10ns cycle interval, the maximum propagation and transition resolution delay is exactly:

$$T = \frac{1}{f} = \frac{1}{100 \times 10^6 \text{ Hz}} = 10 \text{ ns}$$

4.2 Test Cases and Boundary Conditions

To verify system reliability under unpredictable real-world scenarios, the design was subjected to deterministic test cases simulating sequential and conflicting sensor inputs.

Test Case ID	Initial State	Active Inputs	Expected Next State	Simulated Next State	Pass / Fail
TC-01	IDLE (00)	payment = 1, obstruction = 0	OPENING (01)	OPENING (01)	Pass
TC-02	OPENING (01)	Automatic	OPEN (10)	OPEN (10)	Pass
TC-03	OPEN (10)	exit_sensor = 1, obstruction = 0	CLOSING (11)	CLOSING (11)	Pass
TC-04	CLOSING (11)	obstruction = 1, exit_sensor = 0	OPENING (01)	OPENING (01)	Pass
TC-05 (Conflict)	CLOSING (11)	obstruction = 1, exit_sensor = 1	OPENING (01)	OPENING (01)	Pass

4.2.1 Analysis of Conflicting Inputs (TC-05)

In Test Case 05, a conflict scenario was simulated where a exiting vehicle triggers the exit loop sensor to close the gate, but an object simultaneously activates the safety obstruction sensor.

The simulation proved that the obstruction signal functions as a high-priority interrupt over standard operational routing. Because the conditional safety block is positioned outside and prioritized above the normal state-handling case statements inside the sequential block, the synthesized combinational logic network overrides the exit_sensor command. This forces the edge-triggered flip-flops to instantly load the safe OPENING state value on the next clock transition, ensuring an entirely fail-safe operation.

CHAPTER V

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This project successfully achieved the design and verification of a fail-safe automated vehicle barrier system using a synthesizable Verilog Finite State Machine (FSM). Through behavioral simulation in Xilinx Vivado, the following results were confirmed:

- **Logical Integrity:** The FSM correctly transitions between IDLE, OPENING, OPEN, and CLOSING states based on external sensor inputs.
- **Memory Reliability:** The utilization of synchronous D-Flip Flop registers proved highly dependable for state retention, preventing the system from experiencing logical instability or losing its structural position during brief sensor pulses.
- **Safety Efficiency:** The high-priority obstruction override was successfully verified, ensuring that the barrier immediately reverses movement to the OPENING state upon detecting a conflict, thereby mitigating potential mechanical damage or vehicle impact.

5.2 Recommendations

While the digital logic has been verified, the following areas are recommended for future development to transition from simulation to a physical environment:

- **Physical FPGA Integration:** Future work should involve deploying the Verilog code onto physical FPGA hardware (e.g., a Basys 3 or Nexys board) to test real-world signal delays and power consumption.
- **Advanced Timing Analysis:** Utilizing **Vivado Static Timing Analysis (STA)** is recommended to ensure that the logic meets setup and hold time requirements when interfacing with actual high-speed hardware.
- **Hardware Interfacing:** The integration of **Pulse Width Modulation (PWM)** modules should be explored to control the torque and speed of physical servo or DC motors used in the barrier arm mechanism.
- **Sensor Redundancy:** Implementing secondary sensors or a "Watchdog Timer" could further enhance the fail-safe capabilities, ensuring the system resets if a state remains active for an abnormal duration.

References:

[1] L. Dilshan, "UART using Verilog," GitHub.

<https://github.com/LasiduDilshan/UART-using-Verilog?tab=readme-ov-file#how-to-use>

[2] W. Hedfi, "FPGA ADC PWM Motor Control," GitHub.

https://github.com/WassimHedfi/FPGA_ADC_PWM_MotorControl

[3] Devipriya1921, "Traffic Light Controller using Verilog," GitHub.

<https://github.com/Devipriya1921/Traffic-Light-Controller-using-Verilog>

Source Code:

A.1 Barrier Control FSM (Finite State Machine)

- The core logic responsible for state transitions (**IDLE**, **OPENING**, **OPEN**, **CLOSING**) based on the payment and safety sensor inputs.

A.2 UART Module (Proposed Hardware Interface Extension)

- *Note: This reference component outlines the protocol framework for future integration to facilitate communication between the barrier controller and an external payment terminal board.*

A.3 PWM Module (Proposed Mechanical Driver Extension)

- *Note: This reference framework outlines structural motor acceleration methodologies for eventual physical hardware integration to handle arm torque smoothly.*

APPENDICES

Appendix A:

Datasheets

B.1 Synchronous Gating and Propagation Delays

The design adheres to strict Setup (t_{su}) and Hold (t_h) times, ensuring sensor inputs stabilize before the rising edge of the system clock to prevent metastability.

Propagation Delay (t_{pd}): Maximum combinational LUT delay is constrained to stay well within a single clock period.

Clock Frequency: Designed for a 100MHz system clock (10ns period), typical of FPGA development boards like the Xilinx Artix-7.

B.2 Component Specifications (Proposed Hardware Interface)

Infrared (IR) Beam Sensors (exit_sensor, obstruction): * Operating Voltage: 3.3V to 5V DC (FPGA I/O compliant).

Response Time: $< 2ms$ (well within tracking threshold of vehicle movement).

Servo / DC Gear Motor (Barrier Actuator): * Operating Voltage: 5V to 12V DC (isolated from the FPGA via optocouplers or motor driver).

Stall Torque: Minimum 15kg . cm to lift a lightweight barrier arm efficiently.